Phase 2: Multi-Qubit Foundations

Cai Yundi Elliot

Contents

1 Tensor Products and Composite Systems **3**
1.1 Why Tensor Products? The Question of Combining Qubits 3
1.2 The Mathematics: How We Write Combined States 3
1.2.1 Matrix Representation: Computing with Kronecker Product . . . 4
1.2.2 Separable vs Entangled: The Fundamental Distinction 5
1.3 Code: Hello 2-Qubit (Level 11) 6
1.4 Code: Tensor Product Operations (Level 12) 7
1.5 Code: Tensor Product Calculator (Level 13) 8

2 Entanglement and Bell States **10**
2.1 What Is Entanglement? 10
2.2 Bell States: The Four Maximally Entangled States 10
2.3 Code: Create Bell State Phi+ (Level 14) 11
2.4 Code: All Bell States (Level 15) 12
2.5 Code: Bell State Factory (Level 16) 14
2.6 Code: Measure Bell State (Level 17) 16
2.7 Code: Bell State Analyzer (Level 18) 17

3 Measurement and Ancilla **21**
3.1 Measurement as Physical Process 21
3.1.1 The Mathematical Framework 21
3.1.2 General Measurement Theory 22
3.1.3 Real-World Implementations 22
3.1.4 Key Insights 23
3.2 Code: Ancilla Measurement Implementation (Level 19) 23

4 Multi-Qubit Gates **26**
4.1 Why Multi-Qubit Gates? 26
4.1.1 CNOT Gate (Controlled-NOT) 26
4.1.2 A-X-B-X-C Decomposition: Building Any Controlled-U Gate . . 27
4.1.3 Code: Building Controlled-U with A-X-B-X-C (Level 20) 28
4.1.4 CZ Gate (Controlled-Z) 31
4.1.5 SWAP Gate 32
4.1.6 Universality of CNOT 33
4.2 Code Demonstration: Multi-Qubit Gates 33

5 Quantum Teleportation **35**
5.1 What Does Teleportation Achieve? 35
5.2 Protocol Steps: Complete Derivation 35
5.3 Code: Quantum Teleportation Demo 36

6 Superdense Coding **39**
6.1 What Does Superdense Coding Achieve? 39
6.2 Protocol Steps: Complete Derivation 39

7 Exercises **40**

8 Key Physical Concepts **41**

9 Why Multi-Qubit Physics Matters **42**

1 Tensor Products and Composite Systems

1.1 Why Tensor Products? The Question of Combining Qubits

When you first learn about single qubits, a natural question arises: what happens when we have two qubits together? Can we just describe them separately, or do we need something new? The answer is both surprising and profound. When two quantum systems combine, they don't simply sit side by side—they merge into a new mathematical space called the **tensor product space**. This isn't just mathematical convenience; it's how nature actually works.

Think of it this way: if you have two coins, you can describe each one independently. But if those coins were somehow connected so that flipping one always affected the other, you'd need a new way to describe them together. Quantum systems have this connection built into their very description. The tensor product gives us the language to talk about composite systems, and it reveals something remarkable: the combined system has far more possibilities than the individual parts.

Here's where quantum computing draws its power. A single qubit lives in a 2-dimensional space (spanned by $|0\rangle$ and $|1\rangle$). But when you combine two qubits, you don't get 4 qubits—you get a 4-dimensional space. Add a third qubit, and you have 8 dimensions. This exponential growth continues: n qubits create a 2^n-dimensional space. With just 50 qubits, you're working in a space with over 10^{15} dimensions—larger than the number of atoms in the observable universe. This exponential explosion is why quantum computers can tackle problems that would overwhelm any classical machine.

But there's a deeper consequence. When systems combine via tensor product, something new becomes possible: **entanglement**. Not every state in the combined space can be written as "qubit 1 is in state X, qubit 2 is in state Y." Some states are intrinsically connected—they cannot be separated into individual descriptions. These entangled states are the heart of quantum computing's power, enabling teleportation, superdense coding, and computational speedups impossible classically.

The tensor product framework appears throughout physics. In chemistry, molecular orbitals are tensor products of atomic orbitals. In condensed matter physics, spin chains are modeled as tensor products of individual spins. Every multi-qubit quantum processor—from superconducting chips to trapped ions—relies on this structure. Even quantum field theory builds its Fock space from tensor products of mode spaces. This isn't abstract mathematics; it's the architecture of reality at the quantum scale.

1.2 The Mathematics: How We Write Combined States

Now that we understand why tensor products matter, let's learn how to actually write them. The notation is elegant and systematic. For two vector spaces V and W with bases $\{|v_i\rangle\}$ and $\{|w_j\rangle\}$, the tensor product space $V \otimes W$ has basis elements written as $|v_i\rangle \otimes |w_j\rangle$. For qubits, this becomes beautifully concrete.

Suppose you have two qubits: the first in state $|\psi_1\rangle = \alpha|0\rangle + \beta|1\rangle$ and the second in state $|\psi_2\rangle = \gamma|0\rangle + \delta|1\rangle$. When you combine them via tensor product:

$$|\psi_1\rangle \otimes |\psi_2\rangle = (\alpha|0\rangle + \beta|1\rangle) \otimes (\gamma|0\rangle + \delta|1\rangle) = \alpha\gamma|00\rangle + \alpha\delta|01\rangle + \beta\gamma|10\rangle + \beta\delta|11\rangle \quad (1)$$

Each term has a clear meaning: $|00\rangle$ means both qubits in state 0, $|01\rangle$ means first qubit 0 and second qubit 1, etc. The coefficients (like $\alpha\gamma$) are joint probability amplitudes.

Squaring their magnitudes gives measurement probabilities. The total must sum to 1 (normalization).

The exponential growth of dimension is worth pausing to appreciate. One qubit gives you 2 dimensions. Two qubits give you 4. Three give you 8. By the time you reach 10 qubits, you have 1,024 dimensions. At 20 qubits, you're past one million. At 50 qubits, the dimension exceeds 10^{15}. And at around 300 qubits, the dimension rivals the number of atoms in the observable universe. This isn't just impressive—it's why quantum computers can explore solution spaces that would take classical computers longer than the age of the universe to traverse.

1.2.1 Matrix Representation: Computing with Kronecker Product

The tensor product of vectors is computed via the **Kronecker product**. For two vectors $\mathbf{a} = (a_1, a_2)^T$ and $\mathbf{b} = (b_1, b_2)^T$:

$$\mathbf{a} \otimes \mathbf{b} = \begin{pmatrix} a_1 b_1 \\ a_1 b_2 \\ a_2 b_1 \\ a_2 b_2 \end{pmatrix} \tag{2}$$

Numerical example: If $\mathbf{a} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $\mathbf{b} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$:

$$\begin{pmatrix} 1 \\ 0 \end{pmatrix} \otimes \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \cdot 0 \\ 1 \cdot 1 \\ 0 \cdot 0 \\ 0 \cdot 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \tag{3}$$

Computational basis states (matrix representations):

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \tag{4}$$

$$|00\rangle = |0\rangle \otimes |0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \otimes \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} \tag{5}$$

$$|01\rangle = |0\rangle \otimes |1\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \otimes \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \tag{6}$$

$$|10\rangle = |1\rangle \otimes |0\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \otimes \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} \tag{7}$$

$$|11\rangle = |1\rangle \otimes |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \otimes \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} \tag{8}$$

The position in the four-component vector indicates which basis state: position 1 = $|00\rangle$, position 2 = $|01\rangle$, position 3 = $|10\rangle$, position 4 = $|11\rangle$. This ordering (qubit 1 first, then qubit 2) is called "big-endian."

Another numerical example: Tensor product of superposition states:

$$|+\rangle \otimes |0\rangle = \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix} \otimes \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1/\sqrt{2} \\ 0 \\ 1/\sqrt{2} \\ 0 \end{pmatrix} = \frac{|00\rangle + |10\rangle}{\sqrt{2}} \tag{9}$$

This structure enables everything we do with multi-qubit systems. Local operations—acting on just one qubit—are written as $A \otimes I$, where I is the identity on the other qubit. Parallel gates—acting on both qubits simultaneously—are written as $A \otimes B$. Hamiltonians (the operators governing time evolution) often have tensor product structure. Measurements are described as tensor products of projectors. And crucially, operators acting on different qubits always commute, which simplifies analysis tremendously. The exponential scaling is worth noting: 2 qubits need 4 amplitudes, but 10 qubits need 1,024 amplitudes. This is why classical simulation of large quantum systems becomes impossible.

1.2.2 Separable vs Entangled: The Fundamental Distinction

Here we arrive at the most important distinction in multi-qubit quantum mechanics. Some combined states can be written as a simple tensor product: $|\psi\rangle = |\psi_1\rangle \otimes |\psi_2\rangle$. These are called **separable states**, and they describe two qubits that are essentially independent. If you measure one, you learn nothing about the other. Their measurement outcomes are uncorrelated, just like flipping two separate coins. Each qubit has its own complete description, and the joint probability of measurement outcomes factors into the product of individual probabilities. States like $|00\rangle = |0\rangle \otimes |0\rangle$ are separable—they can be prepared by local operations alone, and they behave much like independent classical systems. A classical computer can simulate separable states efficiently by tracking each qubit separately.

But other states cannot be factored this way. No matter how hard you try, you cannot write them as $|\psi_1\rangle \otimes |\psi_2\rangle$. These are **entangled states**, and they represent something profoundly non-classical. The qubits are connected in a way that defies ordinary intuition. Measure one qubit, and you instantly know something about the other—even if they're light-years apart. The measurement outcomes are perfectly correlated (or anti-correlated), and this correlation is stronger than anything possible in classical physics. You cannot describe the individual qubits separately; only the joint system has a complete description. This is the resource that enables quantum teleportation, superdense coding, and computational speedups impossible classically. Entanglement is also fragile—interaction with the environment (decoherence) destroys it, which is why building practical quantum computers is so challenging.

The canonical example is the Bell state $|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$. Try as you might, you cannot factor this into "qubit 1 is in state X, qubit 2 is in state Y." It's intrinsically a joint state. When you measure this state, you'll find either $|00\rangle$ or $|11\rangle$, each with 50% probability. But you'll never find $|01\rangle$ or $|10\rangle$. The qubits always match. This correlation is the signature of entanglement. There's also a subtle property called monogamy: a qubit maximally entangled with one partner cannot be equally entangled with another.

Entanglement is limited in how it can be shared, which has deep implications for quantum cryptography and many-body physics.

1.3 Code: Hello 2-Qubit (Level 11)

File: `code/multi_qubit/11_hello_2qubit.py`

```python
#!/usr/bin/env python3
"""
Level 11: Hello 2-Qubit - Task Specific

Creates a simple 2-qubit state |00>.
Task-specific, but now we're in multi-qubit land!

Learning goal: First step into multi-qubit systems!
"""

from qiskit.quantum_info import Statevector

# Create |00> state (both qubits in ground state)
zero_zero = Statevector.from_label('00')

print("My first 2-qubit state:")
print(zero_zero)

# Notice: 4 components (2^2 = 4 dimensions)
# |00> = [1, 0, 0, 0]
```

Listing 1: Create 2-qubit state (Level 11)

What This Code Does:

This code creates the simplest 2-qubit state $|00\rangle$ using Qiskit's Statevector. Unlike single-qubit states (2 components), the 2-qubit state has 4 components ($2^2 = 4$ dimensions), representing the computational basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$. The output shows $|00\rangle = [1, 0, 0, 0]$, meaning both qubits are in the ground state. This is the first step into multi-qubit systems, where the state space grows exponentially with qubit count.

Physical Significance:

- **Exponential growth**: 2 qubits = 4 dimensions, n qubits = 2^n dimensions
- **Tensor product**: $|00\rangle = |0\rangle \otimes |0\rangle$
- **Separable**: This state is not entangled (product state)
- **Ground state**: Both qubits in lowest energy state
- **Starting point**: Most quantum algorithms begin from $|00\cdots0\rangle$

Expected Output:

```
My first 2-qubit state:
Statevector([1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j],
            dims=(2, 2))
```

1.4 Code: Tensor Product Operations (Level 12)

File: `code/multi_qubit/12_tensor_product.py`

```python
#!/usr/bin/env python3
"""
Level 12: Tensor Product - Task Specific

Computes tensor product of two specific states: |0>  |1> = |01>.
Task-specific (only this one case), but introduces tensor product
    concept.

Learning goal: Understand how to combine qubits!
"""

import numpy as np
from qiskit.quantum_info import Statevector

# Create individual states
psi1 = Statevector.from_label('0')  # |0>
psi2 = Statevector.from_label('1')  # |1>

# Compute tensor product
combined = np.kron(psi1, psi2)  # |0>  |1> = |01>

print("Tensor product: |0>  |1>")
print("Result:", combined)

# Notice: |01> = [0, 1, 0, 0]
# This is how we build multi-qubit states!
```

Listing 2: Tensor product operations (Level 12)

What This Code Does:

This code computes the tensor product $|0\rangle \otimes |1\rangle = |01\rangle$ using NumPy's Kronecker product. It creates two single-qubit states, combines them via tensor product, and shows the resulting 2-qubit state with 4 components. The output $|01\rangle = [0, 1, 0, 0]$ demonstrates how multi-qubit states are constructed from individual qubit states. This is the fundamental operation for building composite quantum systems.

Physical Significance:

- **Tensor product**: Mathematical tool for combining quantum systems
- **State space growth**: 2 qubits $\rightarrow$ 4 dimensions (2^n for n qubits)
- **Separable state**: $|01\rangle = |0\rangle \otimes |1\rangle$ (not entangled)
- **Basis states**: Computational basis built from tensor products
- **Foundation**: All multi-qubit states are linear combinations of tensor products

Expected Output:

```
Tensor product: |0>  |1>
Result: Statevector([0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j],
                    dims=(2, 2))
```

1.5 Code: Tensor Product Calculator (Level 13)

File: `code/multi_qubit/13_tensor_product_calculator.py`

```python
#!/usr/bin/env python3
"""
Level 13: Tensor Product Calculator - Generalized

Generalizes tensor product: any two single-qubit states.
From task-specific to general tool!

Learning goal: Build a reusable tensor product function!
"""

import numpy as np
from qiskit.quantum_info import Statevector


def tensor_product(state1, state2):
    """
    Compute tensor product of two single-qubit states.

    Args:
        state1: First state label ('0', '1', '+', etc.)
        state2: Second state label

    Returns:
        Combined 2-qubit statevector
    """
    # Create individual states
    psi1 = Statevector.from_label(state1)
    psi2 = Statevector.from_label(state2)

    # Compute tensor product
    combined = np.kron(psi1, psi2)

    return combined


# Example 1: |0>  |0> = |00>
result1 = tensor_product('0', '0')
print("1. |0>  |0>:")
print(f"   {result1}")

# Example 2: |0>  |1> = |01>
result2 = tensor_product('0', '1')
print("\n2. |0>  |1>:")
print(f"   {result2}")

# Example 3: |1>  |0> = |10>
result3 = tensor_product('1', '0')
print("\n3. |1>  |0>:")
```

```python
print(f"   {result3}")

# Example 4: |1>   |1> = |11>
result4 = tensor_product('1', '1')
print("\n4. |1>   |1>:")
print(f"   {result4}")

# Example 5: |+>   |0> (superposition   basis)
result5 = tensor_product('+', '0')
print("\n5. |+>   |0>:")
print(f"   {result5}")
```

Listing 3: Tensor product calculator (Level 13)

What This Code Does:

This code generalizes tensor product computation by defining a `tensor_product()` function that accepts any two single-qubit state labels ('0', '1', '+', '-', etc.). It creates individual states, computes their Kronecker product, and returns the combined 2-qubit statevector. The examples demonstrate all computational basis combinations ($|00\rangle$ through $|11\rangle$) plus a superposition case ($|+\rangle \otimes |0\rangle$). This transitions from task-specific (Level 13) to a reusable tool for any tensor product.

Physical Significance:

- **Generalization**: One function handles all 2-qubit product states
- **Composability**: Build complex states from simple components
- **Exponential scaling**: n qubits require 2^n amplitudes
- **Separability**: Product states have no entanglement
- **Progression**: Level 14 generalizes Level 13 (specific $\rightarrow$ general)

Expected Output:

```
1. |0>   |0>:
   Statevector([1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j], dims=(2, 2))

2. |0>   |1>:
   Statevector([0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j], dims=(2, 2))

3. |1>   |0>:
   Statevector([0.+0.j, 0.+0.j, 1.+0.j, 0.+0.j], dims=(2, 2))

4. |1>   |1>:
   Statevector([0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j], dims=(2, 2))

5. |+>   |0>:
   Statevector([0.707+0.j, 0.+0.j, 0.707+0.j, 0.+0.j], dims=(2, 2))
```

2 Entanglement and Bell States

2.1 What Is Entanglement?

A two-qubit state is entangled when it cannot be written as a product of individual qubit states: $|\psi\rangle \neq |\psi_1\rangle \otimes |\psi_2\rangle$. For the Bell state $|\Phi^+\rangle = \frac{|00\rangle+|11\rangle}{\sqrt{2}}$, measurement outcomes are perfectly correlated: measuring one qubit determines the other.

Entanglement is a resource consumed by quantum protocols:

- **Teleportation**: Transfers quantum state using one Bell pair + 2 classical bits
- **Superdense coding**: Sends 2 classical bits using one qubit + one Bell pair
- **QKD**: Guarantees security via entanglement correlations

Entanglement is fragile—decoherence destroys it through environmental coupling.

2.2 Bell States: The Four Maximally Entangled States

The four Bell states form an orthonormal basis for maximally entangled two-qubit states:

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}} \tag{10}$$

$$|\Phi^-\rangle = \frac{|00\rangle - |11\rangle}{\sqrt{2}} \tag{11}$$

$$|\Psi^+\rangle = \frac{|01\rangle + |10\rangle}{\sqrt{2}} \tag{12}$$

$$|\Psi^-\rangle = \frac{|01\rangle - |10\rangle}{\sqrt{2}} \tag{13}$$

Correlation patterns:

- $|\Phi^\pm\rangle$ **(Phi states)**: Both qubits match (00 or 11)
- $|\Psi^\pm\rangle$ **(Psi states)**: Qubits opposite (01 or 10)
- **Plus states ($|\Phi^+\rangle$, $|\Psi^+\rangle$)**: Constructive interference
- **Minus states ($|\Phi^-\rangle$, $|\Psi^-\rangle$)**: Destructive interference for cross terms

Creation circuits:

- $|\Phi^+\rangle$: H on qubit 0, then $\text{CNOT}_{0\to1}$
- $|\Phi^-\rangle$: H + CNOT + Z on one qubit
- $|\Psi^+\rangle$: X + H + CNOT
- $|\Psi^-\rangle$: X + H + CNOT + Z

Properties:

- **Orthonormal**: $\langle\Phi^+|\Phi^-\rangle = 0$, etc. (complete basis)

- **Equally entangled**: All four have maximal entanglement entropy
- **Interconvertible**: Single-qubit gates transform between Bell states
- $|\Psi^-\rangle$ **antisymmetric**: Picks up minus sign under qubit exchange (singlet state)

2.3 Code: Create Bell State Phi+ (Level 14)

File: `code/multi_qubit/14_create_bell_phi_plus.py`

```python
#!/usr/bin/env python3
"""
Level 14: Create Bell State Φ - Task Specific

Creates one specific Bell state: Φ|> = (|00> + |11>)√/2.
Task-specific (only this Bell state), but introduces entanglement!

Learning goal: First entangled state!
"""

from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector

# Create 2-qubit circuit
qc = QuantumCircuit(2)

# Create Bell state Φ|>
qc.h(0)     # Hadamard on qubit 0
qc.cx(0, 1) # CNOT with control=0, target=1

# Get statevector
bell_state = Statevector(qc)

print("Bell state Φ|>:")
print(bell_state)

# This is an ENTANGLED state!
# Can't be written as  |>    |>
# Measurement of one qubit instantly determines the other
```

Listing 4: Create Bell state Φ (Level 14)

What This Code Does:

This code creates the Bell state $|\Phi^+\rangle = \frac{|00\rangle+|11\rangle}{\sqrt{2}}$ using a simple 2-qubit circuit: Hadamard on qubit 0, then CNOT with control=0 and target=1. The circuit generates maximal entanglement between the two qubits. The output shows the statevector with equal amplitudes for $|00\rangle$ and $|11\rangle$ (both $1/\sqrt{2} \approx 0.707$), and zero amplitude for $|01\rangle$ and $|10\rangle$. This is the first entangled state in the roadmap, demonstrating non-separable quantum correlations.

Physical Significance:

- **Entanglement**: Cannot be written as $|\psi_1\rangle \otimes |\psi_2\rangle$

- **Maximal correlation**: Measuring one qubit determines the other instantly
- **Bell basis**: One of 4 maximally entangled 2-qubit states
- **Quantum resource**: Essential for teleportation, superdense coding, QKD
- **Non-locality**: Violates Bell inequalities, stronger than classical correlations

Expected Output:

```
Bell state |Φ>:
Statevector([0.707+0.j, 0.+0.j, 0.+0.j, 0.707+0.j],
            dims=(2, 2))
```

2.4 Code: All Bell States (Level 15)

File: `code/multi_qubit/15_all_bell_states.py`

```python
#!/usr/bin/env python3
"""
Level 15: All Bell States - Task Specific

Creates all 4 Bell states.
Task-specific (only Bell states), but demonstrates all 4 types.

Learning goal: Understand the complete Bell basis!
"""

from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector


def create_bell(bell_type):
    """
    Create one of the 4 Bell states.

    Args:
        bell_type: 'phi_plus', 'phi_minus', 'psi_plus', 'psi_minus'

    Returns:
        Statevector of Bell state
    """
    qc = QuantumCircuit(2)

    if bell_type == 'phi_plus':
        # Φ|> = (|00> + |11>)√/2
        qc.h(0)
        qc.cx(0, 1)

    elif bell_type == 'phi_minus':
        # Φ|> = (|00> - |11>)√/2
        qc.h(0)
```

```
        qc.cx(0, 1)
        qc.z(1)  # Phase flip

    elif bell_type == 'psi_plus':
        # Ψ|> = (|01> + |10>)√/2
        qc.x(0)  # Flip first
        qc.h(0)
        qc.cx(0, 1)

    elif bell_type == 'psi_minus':
        # Ψ|> = (|01> - |10>)√/2
        qc.x(0)
        qc.h(0)
        qc.cx(0, 1)
        qc.z(1)

    else:
        raise ValueError(f"Unknown bell_type: {bell_type}")

    return Statevector(qc)


# Create all 4 Bell states
bell_types = ['phi_plus', 'phi_minus', 'psi_plus', 'psi_minus']

print("All 4 Bell States:\n")

for bell_type in bell_types:
    state = create_bell(bell_type)
    print(f"{bell_type}: {state}")
    print()
```

Listing 5: Create all 4 Bell states (Level 15)

What This Code Does:

This code creates all 4 Bell states using a parameterized `create_bell()` function. Each Bell state is generated by a specific gate sequence: $|\Phi^+\rangle$ (H + CNOT), $|\Phi^-\rangle$ (H + CNOT + Z), $|\Psi^+\rangle$ (X + H + CNOT), and $|\Psi^-\rangle$ (X + H + CNOT + Z). The function demonstrates how single-qubit gates transform between Bell states. This provides complete coverage of the Bell basis, showing all maximally entangled 2-qubit states.

Physical Significance:

- **Bell basis**: Complete orthonormal basis for 2-qubit space
- **Maximal entanglement**: All 4 states are equally entangled
- **Orthogonality**: $\langle\Phi^+|\Phi^-\rangle = 0$, etc.
- **Interconvertible**: Single-qubit gates transform between Bell states
- **Quantum protocols**: Different Bell states used for different applications

Expected Output:

```
All 4 Bell States:

phi_plus: Statevector([0.707+0.j, 0.+0.j, 0.+0.j, 0.707+0.j], dims=(2, 2))
phi_minus: Statevector([0.707+0.j, 0.+0.j, 0.+0.j, -0.707+0.j], dims=(2, 2))
psi_plus: Statevector([0.+0.j, 0.707+0.j, 0.707+0.j, 0.+0.j], dims=(2, 2))
psi_minus: Statevector([0.+0.j, 0.707+0.j, -0.707+0.j, 0.+0.j], dims=(2, 2))
```

2.5 Code: Bell State Factory (Level 16)

File: `code/multi_qubit/16_bell_factory.py`

```python
#!/usr/bin/env python3
"""
Level 16: Bell State Factory - Generalized

Generalizes Bell state creation: any Bell state with parameters.
From task-specific to fully general factory!

Learning goal: Build a parameterized Bell state generator!
"""

from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
import numpy as np


def create_bell(bell_type='phi_plus'):
    """
    Create any of the 4 Bell states.

    Args:
        bell_type: 'phi_plus', 'phi_minus', 'psi_plus', 'psi_minus'

    Returns:
        Statevector and circuit
    """
    qc = QuantumCircuit(2, 2)

    # Start with |00>

    if bell_type == 'phi_plus':
        # Φ|> = (|00> + |11>)√/2
        qc.h(0)
        qc.cx(0, 1)

    elif bell_type == 'phi_minus':
        # Φ|> = (|00> - |11>)√/2
        qc.h(0)
        qc.cx(0, 1)
        qc.z(1)

```

```python
    elif bell_type == 'psi_plus':
        # Ψ|> = (|01> + |10>)√/2
        qc.x(0)
        qc.h(0)
        qc.cx(0, 1)

    elif bell_type == 'psi_minus':
        # Ψ|> = (|01> - |10>)√/2
        qc.x(0)
        qc.h(0)
        qc.cx(0, 1)
        qc.z(1)

    else:
        raise ValueError(f"Unknown bell_type: {bell_type}")

    bell_state = Statevector(qc)
    return bell_state, qc


# Demo: Create all Bell states
print("Bell State Factory Demo\n")

for bell_type in ['phi_plus', 'phi_minus', 'psi_plus', 'psi_minus']:
    state, circuit = create_bell(bell_type)
    print(f"{bell_type.upper()}:")
    print(f"   State: {state}")
    print(f"   Circuit depth: {circuit.depth()} gates")
    print()
```

Listing 6: Bell state factory (Level 16)

What This Code Does:

This code provides a general Bell state factory that creates any of the 4 Bell states based on a parameter. The `create_bell()` function returns both the statevector and the circuit, enabling further analysis. The demo iterates through all Bell types, showing each state and its circuit depth. This transitions from listing all states (Level 16) to a reusable, parameterized generator that can create any Bell state on demand.

Physical Significance:

- **Generalization**: One function creates all 4 Bell states
- **Reusability**: Parameterized design for flexible Bell state generation
- **Circuit analysis**: Returns circuit for depth/gate count analysis
- **Progression**: Level 17 generalizes Level 16 (list → factory)
- **Modularity**: Encapsulates Bell state creation logic

Expected Output:

```
Bell State Factory Demo

PHI_PLUS:
   State: Statevector([0.707+0.j, 0.+0.j, 0.+0.j, 0.707+0.j], dims=(2, 2))
   Circuit depth: 2 gates

PHI_MINUS:
   State: Statevector([0.707+0.j, 0.+0.j, 0.+0.j, -0.707+0.j], dims=(2, 2))
   Circuit depth: 3 gates

PSI_PLUS:
   State: Statevector([0.+0.j, 0.707+0.j, 0.707+0.j, 0.+0.j], dims=(2, 2))
   Circuit depth: 3 gates

PSI_MINUS:
   State: Statevector([0.+0.j, 0.707+0.j, -0.707+0.j, 0.+0.j], dims=(2, 2))
   Circuit depth: 4 gates
```

2.6 Code: Measure Bell State (Level 17)

File: `code/multi_qubit/17_measure_bell.py`

```python
#!/usr/bin/env python3
"""
Level 17: Measure Bell State - Task Specific

Measures Φ|> Bell state and shows perfect correlations.
Task-specific (only Φ), but demonstrates entanglement measurement!

Learning goal: See entanglement in action!
"""

from qiskit import QuantumCircuit, Aer, execute
from qiskit.quantum_info import Statevector


# Create Bell state Φ|>
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)

# Measure
qc.measure_all()

# Run simulation
simulator = Aer.get_backend('qasm_simulator')
result = execute(qc, simulator, shots=1000).result()
counts = result.get_counts()

print("Measuring Φ|> (1000 shots):")
print("="*50)
```

```python

for outcome, count in sorted(counts.items()):
    percentage = count / 1000 * 100
    print(f"   |{outcome}>: {count} ({percentage:.1f}%)")

print("="*50)

# Perfect correlation: only |00> and |11>!
# When you measure one qubit, the other is instantly determined
# This is ENTANGLEMENT!
```

Listing 7: Measure Bell state (Level 17)

What This Code Does:

This code creates the Bell state $|\Phi^+\rangle$, measures both qubits, and runs a simulation with 1000 shots. The output shows measurement statistics with only $|00\rangle$ and $|11\rangle$ outcomes (roughly 50/50), demonstrating perfect correlation. The absence of $|01\rangle$ and $|10\rangle$ outcomes proves the qubits are entangled: measuring one instantly determines the other. This is the first measurement of an entangled state in the roadmap, showing non-classical correlations.

Physical Significance:

- **Perfect correlation**: Only $|00\rangle$ and $|11\rangle$ outcomes (50/50)
- **No $|01\rangle$, $|10\rangle$**: Proof of entanglement
- **Instant determination**: Measuring qubit 0 determines qubit 1
- **Non-locality**: Correlations stronger than classical physics allows
- **Bell inequality**: Violates classical bounds, proves quantum nature

Expected Output:

```
Measuring |Φ> (1000 shots):
==================================================
   |00>: 502 (50.2%)
   |11>: 498 (49.8%)
==================================================
```

2.7 Code: Bell State Analyzer (Level 18)

File: `code/multi_qubit/18_bell_analyzer.py`

```python
#!/usr/bin/env python3
"""
Level 18: Bell State Analyzer - Generalized

Generalizes Bell state measurement: any Bell state, verify entanglement.
From task-specific to general analysis tool!

Learning goal: Build a complete Bell state analysis tool!
```

```python
"""

from qiskit import QuantumCircuit, Aer, execute
from qiskit.quantum_info import Statevector
import numpy as np


def create_bell(bell_type='phi_plus'):
    """Create any Bell state."""
    qc = QuantumCircuit(2, 2)

    if bell_type == 'phi_plus':
        qc.h(0)
        qc.cx(0, 1)
    elif bell_type == 'phi_minus':
        qc.h(0)
        qc.cx(0, 1)
        qc.z(1)
    elif bell_type == 'psi_plus':
        qc.x(0)
        qc.h(0)
        qc.cx(0, 1)
    elif bell_type == 'psi_minus':
        qc.x(0)
        qc.h(0)
        qc.cx(0, 1)
        qc.z(1)

    return qc


def measure_bell(qc, shots=1000):
    """Measure Bell state."""
    simulator = Aer.get_backend('qasm_simulator')
    result = execute(qc, simulator, shots=shots).result()
    return result.get_counts()


def verify_entanglement(counts, bell_type):
    """
    Verify entanglement from measurement results.

    Args:
        counts: Measurement counts
        bell_type: Expected Bell state type

    Returns:
        Boolean: True if entanglement verified
    """
    total = sum(counts.values())

```

```python
    # Phi states: should see 00 and 11 only
    if 'phi' in bell_type:
        correlated = counts.get('00', 0) + counts.get('11', 0)
        fraction = correlated / total
        if fraction > 0.95:
            return True, f"Phi-type correlation: {fraction:.2%}"

    # Psi states: should see 01 and 10 only
    elif 'psi' in bell_type:
        correlated = counts.get('01', 0) + counts.get('10', 0)
        fraction = correlated / total
        if fraction > 0.95:
            return True, f"Psi-type correlation: {fraction:.2%}"

    return False, f"No correlation: {fraction:.2%}"


# Test all Bell states
print("Bell State Analyzer - Full Test\n")

for bell_type in ['phi_plus', 'phi_minus', 'psi_plus', 'psi_minus']:
    qc = create_bell(bell_type)
    counts = measure_bell(qc)

    print(f"{bell_type.upper()}:")
    print(f"   Counts: {counts}")

    verified, message = verify_entanglement(counts, bell_type)
    print(f"   Entanglement: {verified} ({message})")
    print()
```

Listing 8: Bell state analyzer (Level 18)

What This Code Does:

This code provides a complete Bell state analysis tool with three functions: `create_bell()` (creates any Bell state), `measure_bell()` (measures with specified shots), and `verify_entanglement()` (checks correlation pattern). It tests all 4 Bell states, showing measurement counts and verifying entanglement for each. The analyzer confirms Phi states show $|00\rangle+|11\rangle$ correlation and Psi states show $|01\rangle+|10\rangle$ correlation, with >95% fraction proving entanglement. This is the most general Bell state tool, capable of creating, measuring, and verifying any Bell state.

Physical Significance:

- **Complete analysis**: Create, measure, and verify entanglement
- **Correlation pattern**: Phi (00+11) vs Psi (01+10) types
- **Entanglement threshold**: >95% correlation proves quantum nature
- **Progression**: Level 18 generalizes Level 17 (single → all)
- **Verification**: Automated entanglement detection

Expected Output:

```
Bell State Analyzer - Full Test

PHI_PLUS:
   Counts: {'00': 508, '11': 492}
   Entanglement: True (Phi-type correlation: 100.00%)

PHI_MINUS:
   Counts: {'00': 495, '11': 505}
   Entanglement: True (Phi-type correlation: 100.00%)

PSI_PLUS:
   Counts: {'01': 503, '10': 497}
   Entanglement: True (Psi-type correlation: 100.00%)

PSI_MINUS:
   Counts: {'01': 498, '10': 502}
   Entanglement: True (Psi-type correlation: 100.00%)
```

3 Measurement and Ancilla

3.1 Measurement as Physical Process

When you measure a quantum system, what actually happens? The textbooks tell you about wavefunction collapse and the Born rule, but they rarely explain what's physically occurring. Here's the deeper truth: measurement is not some mystical instantaneous event. It's a physical process that can be understood as unitary evolution on a larger system, followed by reading out a result.

3.1.1 The Mathematical Framework

Consider a qubit in state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where $|\alpha|^2 + |\beta|^2 = 1$. We model measurement as physical interaction with an ancilla qubit.

Step 1: Initialize the ancilla. Add a second qubit in state $|0\rangle$. The combined initial state is:

$$|\Psi_0\rangle = |\psi\rangle \otimes |0\rangle = (\alpha|0\rangle + \beta|1\rangle) \otimes |0\rangle = \alpha|00\rangle + \beta|10\rangle \tag{14}$$

Step 2: Apply unitary evolution. Apply CNOT with system as control, ancilla as target:

$$U_{\text{CNOT}}|00\rangle = |00\rangle, \quad U_{\text{CNOT}}|10\rangle = |11\rangle \tag{15}$$

Applying this to our state:

$$|\Psi_1\rangle = U_{\text{CNOT}}|\Psi_0\rangle = \alpha|00\rangle + \beta|11\rangle \tag{16}$$

This is an **entangled state**—system and ancilla are perfectly correlated.

Step 3: Measure the ancilla. Measurement operators are:

$$M_0 = I \otimes |0\rangle\langle 0|, \quad M_1 = I \otimes |1\rangle\langle 1| \tag{17}$$

The probability of outcome 0 is:

$$p(0) = \langle\Psi_1|M_0^\dagger M_0|\Psi_1\rangle = |\alpha|^2 \tag{18}$$

The probability of outcome 1 is:

$$p(1) = \langle\Psi_1|M_1^\dagger M_1|\Psi_1\rangle = |\beta|^2 \tag{19}$$

These are exactly the Born rule probabilities.

Step 4: Post-measurement state. If outcome 0 is observed:

$$|\Psi_2\rangle = \frac{M_0|\Psi_1\rangle}{\sqrt{p(0)}} = \frac{\alpha|00\rangle}{|\alpha|} = |00\rangle \tag{20}$$

If outcome 1 is observed:

$$|\Psi_2\rangle = \frac{M_1|\Psi_1\rangle}{\sqrt{p(1)}} = \frac{\beta|11\rangle}{|\beta|} = |11\rangle \tag{21}$$

The system qubit collapses to match the ancilla measurement outcome. Measuring the ancilla indirectly measures the system.

3.1.2 General Measurement Theory

This specific example illustrates a profound general result. Any measurement described by measurement operators $\{M_m\}$ satisfying $\sum_m M_m^\dagger M_m = I$ can be implemented as:

1. Start with system $|\psi\rangle$ and ancilla $|0\rangle$
2. Apply unitary U such that: $U|\psi\rangle|0\rangle = \sum_m M_m|\psi\rangle|m\rangle$
3. Measure ancilla in computational basis
4. Outcome m occurs with probability $p(m) = \langle\psi|M_m^\dagger M_m|\psi\rangle$
5. System collapses to $\frac{M_m|\psi\rangle}{\sqrt{p(m)}}$

This is the **measurement dilation theorem** (also called Neumark's theorem or Naimark extension). It tells us that every generalized measurement (POVM) can be realized as a projective measurement on a larger Hilbert space.

For our CNOT-based Z-measurement, the measurement operators are $M_0 = |0\rangle\langle 0|$ and $M_1 = |1\rangle\langle 1|$ (projective Z-basis measurement). The unitary $U = \text{CNOT}$ satisfies:

$$\text{CNOT} \cdot (\alpha|0\rangle + \beta|1\rangle) \otimes |0\rangle = \alpha|00\rangle + \beta|11\rangle = M_0|\psi\rangle|0\rangle + M_1|\psi\rangle|1\rangle$$

3.1.3 Real-World Implementations

This isn't just mathematical formalism—it's how physical quantum measurements actually work:

- **Superconducting qubits**: Microwave resonator dispersively coupled to qubit. The qubit state shifts the resonator frequency; measuring the resonator (ancilla) reveals the qubit state.
- **Trapped ions**: Fluorescence photons emitted when ion is in a particular state. The photon detection (ancilla readout) indicates the ion's internal state.
- **Quantum dots**: Charge sensor detects electron spin state via spin-to-charge conversion. The charge sensor acts as ancilla.
- **Photonics**: Beam splitter + detector where path acts as ancilla. Detecting which path the photon took constitutes measurement.
- **Error correction**: Syndrome extraction uses ancilla qubits entangled with data qubits. Measuring ancillas reveals error syndromes without collapsing data.

Every physical measurement involves coupling your system to something larger, creating entanglement, and then reading out the larger system.

3.1.4 Key Insights

The measurement dilation theorem has three profound implications:

- **Unitary evolution**: No collapse occurs during the system-ancilla interaction. The combined evolution is fully unitary and reversible (until ancilla readout).
- **Entanglement**: The CNOT creates perfect correlations between system and ancilla. This entanglement is the mechanism by which ancilla measurement reveals system information.
- **Universality**: *Any* measurement—projective, weak, or generalized (POVM)—can be implemented this way by choosing appropriate unitary U and ancilla dimension.

This framework reveals that measurement isn't fundamentally different from other quantum operations. It's unitary evolution plus entanglement plus readout. The apparent "collapse" is simply the update of our knowledge after learning the measurement outcome—a Bayesian update, not a physical discontinuity.

3.2 Code: Ancilla Measurement Implementation (Level 19)

File: `code/multi_qubit/19_ancilla_measurement.py`

```python
#!/usr/bin/env python3
"""
Ancilla-Based Measurement Implementation
Demonstrates measurement as unitary evolution + ancilla readout

Measurement as unitary evolution: U|psi>|0> = Sum M_m|psi>|m>
"""
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit_aer import AerSimulator
import numpy as np

sim = AerSimulator()

# Create |psi>|0> where |psi> = 0.6|0> + 0.8|1>
# Qiskit ordering: |q1 q0>, so we want 0.6|00> + 0.8|01>
qc = QuantumCircuit(2, 1)

# Method: Start with |00>, apply rotation to qubit 0
alpha, beta = 0.6, 0.8
theta = 2 * np.arccos(alpha)  # cos(theta/2) = alpha

qc.ry(theta, 0)  # Rotate qubit 0 to alpha|0> + beta|1>

print("=" * 60)
print("ANCIlla-BASED MEASUREMENT")
print("=" * 60)

print("\n1. Initial state |psi>|0>:")
```

```python
sv_initial = Statevector(qc)
print(f"   Statevector: {[f'{x:.2f}' for x in sv_initial.data]}")
print(f"   = 0.60|00> + 0.80|01>")
print(f"   = (0.6|0> + 0.8|1>) tensor |0>")
print(f"   (qubit 0 is in superposition, qubit 1 is |0>)")

# Apply CNOT: control=qubit 0, target=qubit 1
# This entangles the system: |00> -> |00>, |01> -> |11>
qc.cx(0, 1)

print("\n2. After unitary U (CNOT with control=0, target=1):")
sv_entangled = Statevector(qc)
print(f"   Statevector: {[f'{x:.2f}' for x in sv_entangled.data]}")
print(f"   = 0.60|00> + 0.80|11>  [ENTANGLED!]")
print(f"   = M0|psi>|0> + M1|psi>|1>")
print(f"   where M0=|0><0|, M1=|1><1| (projective measurement)")

# Measure ancilla (qubit 1)
qc.measure(1, 0)

print("\n3. Measurement statistics (1000 shots):")
job = sim.run(qc, shots=1000)
result = job.result()
counts = result.get_counts()
print(f"   Counts: {counts}")

# Calculate expected probabilities
p0 = abs(0.6)**2  # 0.36
p1 = abs(0.8)**2  # 0.64

print(f"\n4. Comparison with theory (Born rule):")
print(f"   P(0) = |0.6|^2 = {p0:.4f} (expected ~{int(p0*1000)}
    counts)")
print(f"   P(1) = |0.8|^2 = {p1:.4f} (expected ~{int(p1*1000)}
    counts)")

# Show post-measurement states
print(f"\n5. Post-measurement states (collapse):")
print(f"   If outcome '0': system collapses to |0>")
print(f"      (normalized M0|psi> = |0><0|psi>/sqrt(p0) = 0.6|0>/0.6
     = |0>)")
print(f"   If outcome '1': system collapses to |1>")
print(f"      (normalized M1|psi> = |1><1|psi>/sqrt(p1) = 0.8|1>/0.8
     = |1>)")

print("\n" + "=" * 60)
print("KEY INSIGHT: Measurement = Unitary + Ancilla Readout")
print("This proves any measurement can be implemented as unitary")
print("evolution on a larger system!")
print("=" * 60)
```

```
print("\nFORMULA SUMMARY:")
print("  Probability:      p(m) = <psi|M_m^dag M_m|psi>")
print("  Post-measurement: |psi'> = M_m|psi> / sqrt(p(m))")
print("  For projective Z: M_0=|0><0|, M_1=|1><1|")
```

Listing 9: Ancilla-based measurement (Level 19)

What This Code Does:

This code demonstrates ancilla-based measurement by preparing a superposition state $|\psi\rangle = 0.6|0\rangle + 0.8|1\rangle$, adding an ancilla qubit in $|0\rangle$, applying CNOT to entangle them, and measuring the ancilla. The CNOT creates the entangled state $0.6|00\rangle + 0.8|11\rangle$, correlating system and ancilla. Measuring the ancilla collapses the system to $|0\rangle$ or $|1\rangle$ with probabilities matching the Born rule (36% and 64%). This proves any measurement can be implemented as unitary evolution on a larger system.

Physical Significance:

- **Unitary evolution**: CNOT creates entanglement without collapse
- **Entanglement**: System and ancilla become perfectly correlated
- **Ancilla readout**: Measuring ancilla indirectly measures system
- **Born rule**: Probabilities match $|\alpha|^2$ and $|\beta|^2$
- **Collapse**: System state collapses based on ancilla outcome
- **Universality**: Any measurement can be implemented this way

Expected Output:

```
============================================================
ANCIlla-BASED MEASUREMENT
============================================================

1. Initial state |psi>|0>:
   Statevector: ['0.60', '0.80', '0.00', '0.00']
   = 0.60|00> + 0.80|01>

2. After CNOT (entangled state):
   Statevector: ['0.60', '0.00', '0.00', '0.80']
   = 0.60|00> + 0.80|11>  [ENTANGLED!]

3. Measurement statistics (1000 shots):
   Counts: {'0': 362, '1': 638}

4. Comparison with theory:
   P(0) = |0.6|^2 = 0.3600 (expected ~360 counts)
   P(1) = |0.8|^2 = 0.6400 (expected ~640 counts)

5. Post-measurement states:
   If outcome '0': system collapses to |0>
   If outcome '1': system collapses to |1>
```

4 Multi-Qubit Gates

4.1 Why Multi-Qubit Gates?

Single-qubit gates are powerful, but they have a fundamental limitation: they cannot create entanglement. If you start with a separable state and apply only single-qubit gates, you'll always end with a separable state. To unlock the full power of quantum computing, you need gates that act on multiple qubits simultaneously. These multi-qubit gates are the engines of entanglement, and they enable everything from quantum algorithms to error correction.

4.1.1 CNOT Gate (Controlled-NOT)

The CNOT gate flips the target qubit if and only if the control qubit is in state $|1\rangle$.

Matrix representation (control = qubit 0, target = qubit 1):

$$\text{CNOT}_{0\to1} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \tag{22}$$

This matrix acts on the computational basis ordered as $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$. The action is:

$$\text{CNOT}_{0\to1}|00\rangle = |00\rangle \tag{23}$$
$$\text{CNOT}_{0\to1}|01\rangle = |01\rangle \tag{24}$$
$$\text{CNOT}_{0\to1}|10\rangle = |11\rangle \tag{25}$$
$$\text{CNOT}_{0\to1}|11\rangle = |10\rangle \tag{26}$$

When control (qubit 0) is $|0\rangle$, target is unchanged. When control is $|1\rangle$, target is flipped.

Matrix representation (control = qubit 1, target = qubit 0):

$$\text{CNOT}_{1\to0} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \tag{27}$$

The action is:

$$\text{CNOT}_{1\to0}|00\rangle = |00\rangle \tag{28}$$
$$\text{CNOT}_{1\to0}|01\rangle = |11\rangle \tag{29}$$
$$\text{CNOT}_{1\to0}|10\rangle = |10\rangle \tag{30}$$
$$\text{CNOT}_{1\to0}|11\rangle = |01\rangle \tag{31}$$

When control (qubit 1) is $|0\rangle$, target is unchanged. When control is $|1\rangle$, target is flipped.

Creating entanglement with CNOT:

Start with $|00\rangle$, apply Hadamard to the first qubit:

$$|+0\rangle = H \otimes I \cdot |00\rangle = \frac{|00\rangle + |10\rangle}{\sqrt{2}} \tag{32}$$

Then apply CNOT:

$$\text{CNOT}_{0\to1} \cdot \frac{|00\rangle + |10\rangle}{\sqrt{2}} = \frac{|00\rangle + |11\rangle}{\sqrt{2}} = |\Phi^+\rangle \tag{33}$$

The CNOT transforms a product state into the Bell state $|\Phi^+\rangle$ (maximally entangled).

4.1.2 A-X-B-X-C Decomposition: Building Any Controlled-U Gate

Any single-qubit unitary U can be decomposed into three rotations:

$$U = R_z(\beta)R_y(\gamma)R_z(\delta) \tag{34}$$

where γ, β, δ are extracted from $U = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$:

$$\gamma = 2\arccos(|a|), \quad \beta, \delta \text{ from phases of } a, b, c, d \tag{35}$$

Controlled-U construction: Define three single-qubit gates A, B, C:

$$A = R_z(\beta)R_y(\gamma/2) \tag{36}$$
$$B = R_y(-\gamma/2)R_z(-(\beta+\delta)/2) \tag{37}$$
$$C = R_z((\delta-\beta)/2) \tag{38}$$

These satisfy two key properties:

$$ABC = I \quad \text{(identity)} \tag{39}$$

$$AXBXC = U \quad \text{(target unitary)} \tag{40}$$

Controlled-U circuit: Place A, B, C on the target qubit with CNOT gates controlled by the control qubit:

- Apply A to target
- CNOT (control $\to$ target)
- Apply B to target
- CNOT (control $\to$ target)
- Apply C to target

When control is $|0\rangle$: The sequence is $ABC = I$ (target unchanged).
When control is $|1\rangle$: Both CNOTs apply X, giving $AXBXC = U$ (target transformed by U).

Example: Controlled-Hadamard. For $H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$:

$$\gamma = \frac{\pi}{2}, \quad \beta = 0, \quad \delta = \pi \tag{41}$$

$$A = R_z(0)R_y(\pi/4) = R_y(\pi/4) \tag{42}$$
$$B = R_y(-\pi/4)R_z(-\pi/2) \tag{43}$$
$$C = R_z(\pi/2) \tag{44}$$

Verify: $ABC = I$ and $AXBXC = H$.

4.1.3 Code: Building Controlled-U with A-X-B-X-C (Level 20)

File: `code/multi_qubit/20_controlled_u_decomposition.py`

```python
#!/usr/bin/env python3
"""
Level 20: Controlled-U Decomposition - A-X-B-X-C Method

Builds any controlled-U gate using CNOT + single-qubit rotations.

Learning goal: Universal controlled gate construction!
"""

from qiskit import QuantumCircuit
from qiskit.quantum_info import Operator
import numpy as np


def zyz_decomposition(U):
    """
    Extract angles gamma, beta, delta from unitary matrix U.

    U = R_z(beta) R_y(gamma) R_z(delta)
    """
    a, b = U[0, 0], U[0, 1]

    # gamma from |a| = |cos(gamma/2)|
    gamma = 2 * np.arccos(np.abs(a))

    # beta, delta from phases
    if np.abs(gamma) < 1e-10:
        # Diagonal case (Z, S, T)
        beta = 0
        delta = np.angle(U[1, 1]) - np.angle(a)
    elif np.abs(gamma - np.pi) < 1e-10:
        # Off-diagonal case (X, Y)
        beta = np.angle(U[1, 0])
        delta = np.angle(b)
    else:
        # General case (H)
        beta = np.angle(a) - np.angle(b)
```

```
        delta = np.angle(a) + np.angle(b)

    return gamma, beta, delta


def build_controlled_u(U_target):
    """
    Build controlled-U using A-X-B-X-C decomposition.

    Returns: 2-qubit circuit (qubit 0 = control, qubit 1 = target)
    """
    # Extract angles
    gamma, beta, delta = zyz_decomposition(U_target)

    # Build A, B, C circuits
    A = QuantumCircuit(1)
    A.rz(beta, 0)
    A.ry(gamma / 2, 0)

    B = QuantumCircuit(1)
    B.ry(-gamma / 2, 0)
    B.rz(-(beta + delta) / 2, 0)

    C = QuantumCircuit(1)
    C.rz((delta - beta) / 2, 0)

    # Build controlled-U circuit
    qc = QuantumCircuit(2)
    qc.compose(A, [1], inplace=True)  # A on target
    qc.cx(0, 1)                        # CNOT
    qc.compose(B, [1], inplace=True)  # B on target
    qc.cx(0, 1)                        # CNOT
    qc.compose(C, [1], inplace=True)  # C on target

    return qc


# Example 1: Controlled-Hadamard
print("=" * 60)
print("CONTROLLED-HADAMARD")
print("=" * 60)

H = np.array([[1, 1], [1, -1]]) / np.sqrt(2)
print("\nTarget Hadamard matrix:")
print(H)

gamma, beta, delta = zyz_decomposition(H)
print(f"\nExtracted angles:")
print(f"  gamma = {gamma:.4f} rad ({gamma/np.pi:.2f })")
print(f"  beta  = {beta:.4f} rad ({beta/np.pi:.2f })")
print(f"  delta = {delta:.4f} rad ({delta/np.pi:.2f })")
```

```python

qc_ch = build_controlled_u(H)
print("\nControlled-Hadamard circuit:")
print(qc_ch.draw())

# Verify
op = Operator(qc_ch)
U_built = op.data
print("\nBuilt unitary (4x4 matrix):")
print(np.round(U_built, 3))

# Example 2: Controlled-Z rotation
print("\n" + "=" * 60)
print("CONTROLLED-Z ROTATION (R_z (/3))")
print("=" * 60)

theta = np.pi / 3
Rz = np.array([[1, 0], [0, np.exp(1j * theta)]])
print(f"\nTarget R_z({theta:.4f}):")
print(Rz)

qc_cz = build_controlled_u(Rz)
print("\nControlled-R_z circuit:")
print(qc_cz.draw())
```

Listing 10: Build any controlled-U gate (Level 20)

What This Code Does:

This code implements the A-X-B-X-C decomposition to build any controlled-U gate. The `zyz_decomposition()` function extracts angles γ, β, δ from a target unitary U. The `build_controlled_u()` function constructs A, B, C gates and assembles the controlled-U circuit using two CNOTs. The example demonstrates controlled-Hadamard and controlled-$R_z(\pi/3)$, verifying the built unitaries match the targets.

Physical Significance:

- **Universality**: Any controlled-U gate built from CNOT + single-qubit rotations
- **Decomposition**: Z-Y-Z decomposition sufficient for any single-qubit unitary
- **Efficiency**: Only 2 CNOTs needed regardless of target U
- **Modularity**: A, B, C gates are reusable building blocks
- **Verification**: Matrix comparison confirms correctness

Expected Output:

```
============================================================
CONTROLLED-HADAMARD
============================================================

Target Hadamard matrix:
[[ 0.707  0.707]
```

```
 [ 0.707 -0.707]]

Extracted angles:
  gamma = 1.5708 rad (0.50 )
  beta  = 0.0000 rad (0.00 )
  delta = 3.1416 rad (1.00 )

Controlled-Hadamard circuit:

q_0:  0
        CX (0,1)
q_1:  1 RY( /4)  0
              CX(0,1)
                  1


Built unitary (4x4 matrix):
[[ 1.+0.j  0.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j  0.707+0.j  0.+0.j  0.707+0.j]
 [ 0.+0.j  0.+0.j  1.+0.j  0.+0.j]
 [ 0.+0.j  0.707+0.j  0.+0.j -0.707+0.j]]

============================================================
CONTROLLED-Z ROTATION (R_z( /3))
============================================================

Target R_z(1.0472):
[[1.+0.j    0.+0.j   ]
 [0.+0.j    0.5+0.866j]]

Controlled-R_z circuit:

q_0:  0
        CX (0,1)
q_1:  1 RZ(- /12)  0
```

4.1.4 CZ Gate (Controlled-Z)

The CZ gate applies a phase flip (Z gate) to the target qubit if the control is $|1\rangle$. CZ is symmetric—control and target are interchangeable.

Matrix representation:

$$\mathrm{CZ} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \tag{45}$$

The action is:

$$\text{CZ}|00\rangle = |00\rangle \tag{46}$$
$$\text{CZ}|01\rangle = |01\rangle \tag{47}$$
$$\text{CZ}|10\rangle = |10\rangle \tag{48}$$
$$\text{CZ}|11\rangle = -|11\rangle \tag{49}$$

Only the $|11\rangle$ component picks up a phase of -1.
Relationship to CNOT:

$$\text{CZ}_{01} = (I \otimes H) \cdot \text{CNOT}_{0\to1} \cdot (I \otimes H) \tag{50}$$

You can convert CNOT to CZ by surrounding the target with Hadamard gates.

4.1.5 SWAP Gate

The SWAP gate exchanges the states of two qubits.
Matrix representation:

$$\text{SWAP} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{51}$$

The action is:

$$\text{SWAP}|00\rangle = |00\rangle \tag{52}$$
$$\text{SWAP}|01\rangle = |10\rangle \tag{53}$$
$$\text{SWAP}|10\rangle = |01\rangle \tag{54}$$
$$\text{SWAP}|11\rangle = |11\rangle \tag{55}$$

Decomposition into CNOTs:

$$\text{SWAP} = \text{CNOT}_{0\to1} \cdot \text{CNOT}_{1\to0} \cdot \text{CNOT}_{0\to1} \tag{56}$$

SWAP can be built from three CNOT gates, showing CNOT is more fundamental. The circuit:

- CNOT with qubit 0 as control, qubit 1 as target
- CNOT with qubit 1 as control, qubit 0 as target
- CNOT with qubit 0 as control, qubit 1 as target

4.1.6 Universality of CNOT

Together with arbitrary single-qubit gates, CNOT forms a **universal gate set**: any quantum computation can be decomposed into CNOT gates and single-qubit rotations. This is why CNOT is the workhorse of quantum circuits—it's the minimal multi-qubit gate needed for universal quantum computation.

The key properties that make CNOT universal:

- **Entanglement creation**: CNOT can transform product states into entangled states
- **Conditional logic**: Implements "if-then" operations essential for algorithms
- **Reversibility**: CNOT is its own inverse (applying twice returns to original state)
- **Interconvertibility**: Related to CZ, enables different gate decompositions

4.2 Code Demonstration: Multi-Qubit Gates

Note: This code demonstrates CNOT, CZ, and SWAP gates. These gates are used throughout Levels 14-19 (Bell state creation, ancilla measurement, etc.).

```python
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
import numpy as np

# Create 2-qubit circuit
qc = QuantumCircuit(2)

# Initialize to |00>
print("Initial state:", Statevector(qc))

# Apply Hadamard to first qubit
qc.h(0)
print("\nAfter H on qubit 0:")
print(Statevector(qc))  # Should be |+0>

# Apply CNOT
qc.cx(0, 1)
print("\nAfter CNOT:")
print(Statevector(qc))  # Should be Φ|+>

# Verify entanglement
from qiskit.quantum_info import DensityMatrix
rho = DensityMatrix(qc)
print("\nReduced density matrix of qubit 0:")
print(rho.reduce([0]))  # Should be I/2 (maximally mixed)

# Create SWAP
qc_swap = QuantumCircuit(2)
qc_swap.swap(0, 1)
print("\nSWAP gate:")
```

```python
print(qc_swap.draw())
```

Listing 11: Apply multi-qubit gates

What This Code Demonstrates:

This code demonstrates the three fundamental multi-qubit gates: CNOT (creates entanglement), and SWAP (state exchange). The code shows how CNOT transforms $|+0\rangle$ into $|\Phi^+\rangle$ (Bell state), proving entanglement creation. The reduced density matrix calculation shows that individual qubits appear maximally mixed ($I/2$) even though the composite state is pure - a signature of entanglement. These gates appear throughout the code levels: CNOT in Bell state creation (Levels 14-16), ancilla measurement (Level 19), and Bell measurement (Levels 17-18).

Physical Significance:

- **CNOT**: Creates entanglement from product states, implements conditional logic
- **Entanglement signature**: Reduced density matrix is maximally mixed ($I/2$)
- **Pure vs mixed**: Composite state pure, individual qubits mixed (entanglement)
- **Universality**: CNOT + single-qubit gates = universal quantum computation
- **SWAP**: Exchanges quantum states, enables qubit routing in processors
- **Decomposition**: SWAP = 3 CNOTs (shows CNOT primacy)

Expected Output:

```
Initial state: Statevector([1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j], dims=(2, 2))

After H on qubit 0:
Statevector([0.707+0.j, 0.+0.j, 0.707+0.j, 0.+0.j], dims=(2, 2))
= |+0>

After CNOT:
Statevector([0.707+0.j, 0.+0.j, 0.+0.j, 0.707+0.j], dims=(2, 2))
= |Φ+> (Bell state, ENTANGLED!)

Reduced density matrix of qubit 0:
[[0.5+0.j 0.+0.j]
 [0.+0.j 0.5+0.j]]
= I/2 (maximally mixed - proves entanglement!)

SWAP gate:

q_0:  X

q_1:  X
```

5 Quantum Teleportation

5.1 What Does Teleportation Achieve?

Quantum teleportation sounds like science fiction, but it's a real protocol that has been demonstrated in laboratories around the world. The goal is to transfer a quantum state from one location to another without physically sending the qubit itself. This might seem impossible—after all, how can you move something without moving it? The answer lies in entanglement and classical communication.

Here's the key insight: teleportation doesn't transfer matter or energy. It transfers *information*—specifically, the quantum state that describes a qubit. The protocol requires three ingredients: an entangled pair shared between sender (Alice) and receiver (Bob), a Bell measurement performed by Alice, and two classical bits sent from Alice to Bob. When Alice performs her Bell measurement, she destroys the original state (consistent with the no-cloning theorem), but Bob can reconstruct it perfectly using the classical information she sends.

This isn't faster-than-light communication. Alice's measurement outcomes are random, and Bob can't do anything useful until he receives her classical bits, which travel at most at the speed of light. But the protocol is remarkable: it transfers a complete quantum state—including all the quantum information that would require infinite classical bits to describe—using only two classical bits and one shared entangled pair.

Teleportation is foundational for quantum networks and distributed quantum computing. It enables quantum repeaters that can extend the range of quantum communication, and it's a key primitive in many quantum error correction schemes. Understanding teleportation is understanding how quantum information can be manipulated and moved in ways that have no classical analog.

5.2 Protocol Steps: Complete Derivation

Initial state: Alice wants to teleport $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ to Bob. They share a Bell pair $|\Phi^+\rangle = \frac{|00\rangle+|11\rangle}{\sqrt{2}}$, with Alice holding qubit A (her half of the Bell pair) and Bob holding qubit B.

The combined three-qubit state is:

$$|\Psi_0\rangle = |\psi\rangle \otimes |\Phi^+\rangle = (\alpha|0\rangle + \beta|1\rangle) \otimes \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

Step 1: Alice applies CNOT with her state qubit as control and her Bell half as target. This entangles Alice's two qubits:

$$|\Psi_1\rangle = \frac{\alpha|000\rangle + \alpha|011\rangle + \beta|110\rangle + \beta|101\rangle}{\sqrt{2}}$$

Step 2: Alice applies Hadamard to her state qubit, changing to the X-basis for Bell measurement:

$$|\Psi_2\rangle = \frac{1}{2}[\alpha(|0\rangle + |1\rangle)|00\rangle + \alpha(|0\rangle + |1\rangle)|11\rangle + \beta(|0\rangle - |1\rangle)|10\rangle + \beta(|0\rangle - |1\rangle)|01\rangle]$$

Step 3: Alice measures her two qubits in the computational basis. There are four possible outcomes, each with 25% probability:

- 00: Bob's state is $\alpha|0\rangle + \beta|1\rangle$ (original $|\psi\rangle$)
- 01: Bob's state is $\alpha|1\rangle + \beta|0\rangle$ (needs X correction)
- 10: Bob's state is $\alpha|0\rangle - \beta|1\rangle$ (needs Z correction)
- 11: Bob's state is $\alpha|1\rangle - \beta|0\rangle$ (needs ZX correction)

Step 4: Alice sends her 2-bit measurement outcome to Bob via classical channel.
Step 5: Bob applies the appropriate correction based on Alice's message:

- 00: Do nothing (already has $|\psi\rangle$)
- 01: Apply X (bit flip)
- 10: Apply Z (phase flip)
- 11: Apply ZX (both corrections)

Outcome: Bob's qubit is now exactly $|\psi\rangle$. Alice's original state has been destroyed (no-cloning preserved), and the quantum information has been transferred using entanglement and two classical bits.

5.3 Code: Quantum Teleportation Demo

Note: This code demonstrates the complete teleportation protocol. No dedicated file—concepts integrate Levels 14-18 (Bell states, measurement, corrections).

```python
from qiskit import QuantumCircuit, Aer, execute
from qiskit.visualization import plot_state_city
import numpy as np

def teleportation_circuit():
    """Create quantum teleportation circuit."""
    # 3 qubits: 0 = Alice's state, 1 = Alice's Bell half, 2 = Bob's Bell half
    qc = QuantumCircuit(3, 2)  # 3 qubits, 2 classical bits

    # Prepare unknown state on qubit 0 (for demo, we use |+>)
    qc.h(0)

    # Create Bell pair between Alice (1) and Bob (2)
    qc.h(1)
    qc.cx(1, 2)

    # Alice's Bell measurement
    qc.cx(0, 1)
    qc.h(0)
    qc.measure([0, 1], [0, 1])

    # Bob's corrections (controlled by measurement results)
    qc.z(2).c_if(1, 1)  # If bit 1 is 1, apply Z
    qc.x(2).c_if(0, 1)  # If bit 0 is 1, apply X
```

```python

    return qc

# Create circuit
qc = teleportation_circuit()
print("Teleportation circuit:")
print(qc.draw())

# Simulate
simulator = Aer.get_backend('aer_simulator')
result = execute(qc, simulator).result()
counts = result.get_counts()
print("\nMeasurement outcomes:", counts)
# Should be roughly uniform (25% each)
```

Listing 12: Implement quantum teleportation

What This Code Does:

This code implements the complete quantum teleportation protocol using 3 qubits: qubit 0 (Alice's state to teleport), qubit 1 (Alice's half of Bell pair), and qubit 2 (Bob's half). The circuit first creates a Bell pair between Alice and Bob (H + CNOT), then Alice performs Bell measurement on her two qubits (CNOT + H + measure), and finally Bob applies conditional corrections (Z and X gates controlled by measurement results). The simulation shows all four Bell measurement outcomes with roughly equal probability (25% each), demonstrating that the protocol succeeds regardless of which outcome occurs.

Physical Significance:

- **State transfer**: Quantum state moves from Alice to Bob without physical transmission
- **Entanglement resource**: Consumes one Bell pair (pre-shared entanglement)
- **Classical channel**: Requires 2 classical bits (measurement outcome)
- **No-cloning**: Alice's original state destroyed (consistent with no-cloning theorem)
- **Causality**: No faster-than-light communication (classical bits required)
- **Universality**: Works for *any* unknown quantum state
- **Quantum internet**: Foundation for distributed quantum computing
- **Error correction**: Teleportation-based gates in fault-tolerant architectures

Expected Output:

```
Teleportation circuit:

q_0:  H     M

q_1:  H   X   M

q_2:
```

```
Measurement outcomes: {'00': 250, '01': 250, '10': 250, '11': 250}
(roughly 25% each - all four Bell outcomes equally likely)
```

Key Insights:

- **Bell measurement**: Projects Alice's qubits onto Bell basis (4 outcomes)
- **Random outcome**: Each outcome 25% probability (quantum randomness)
- **Bob's state**: Collapses to corrected version based on outcome
- **Conditional gates**: X applied if bit $0 = 1$, Z applied if bit $1 = 1$
- **Verification**: Bob's final state matches Alice's original (teleportation successful)
- **No information leak**: Measurement outcomes random (no state information)

6 Superdense Coding

6.1 What Does Superdense Coding Achieve?

If teleportation sends one qubit using two classical bits, you might wonder: is there a reverse protocol? Superdense coding answers this beautifully: it sends **two classical bits** using **one qubit**, provided you have pre-shared entanglement. This doubling of classical capacity is impossible without entanglement—it's a uniquely quantum advantage.

The protocol is elegantly simple. Alice and Bob start by sharing a Bell pair. Alice wants to send two classical bits (00, 01, 10, or 11). Depending on her message, she applies a specific gate to her half of the Bell pair: nothing for 00, X for 01, Z for 10, or ZX for 11. These operations transform the shared Bell state into one of the four Bell states, effectively encoding her two-bit message. Alice then sends her single qubit to Bob. Bob, now holding both qubits, performs a Bell measurement that distinguishes all four Bell states, perfectly recovering Alice's two-bit message.

The "dense" in superdense coding refers to packing two classical bits of information into one transmitted qubit. Without entanglement, one qubit can carry at most one classical bit (Holevo's bound). But with entanglement as a resource, the capacity doubles. This is the time-reversed dual of teleportation: teleportation consumes two classical bits to send one qubit, while superdense coding consumes one qubit to send two classical bits. Both protocols rely on the same ingredients—Bell pairs, Bell measurements, and the interconvertibility of Bell states via single-qubit gates.

Superdense coding demonstrates that entanglement is a communication resource. It has applications in quantum cryptography (quantum steganography) and provides fundamental insights into the nature of quantum information. While less practically important than teleportation, it's a beautiful demonstration of how entanglement enables tasks impossible classically.

6.2 Protocol Steps: Complete Derivation

Initial state: Alice and Bob share $|\Phi^+\rangle = \frac{|00\rangle+|11\rangle}{\sqrt{2}}$, with Alice holding qubit A and Bob holding qubit B.

Step 1: Alice wants to send two classical bits. She encodes her message by applying gates to her qubit:

Message	Gates	Resulting State
00	I (do nothing)	$\|\Phi^+\rangle = \frac{\|00\rangle+\|11\rangle}{\sqrt{2}}$
01	X	$\|\Psi^+\rangle = \frac{\|01\rangle+\|10\rangle}{\sqrt{2}}$
10	Z	$\|\Phi^-\rangle = \frac{\|00\rangle-\|11\rangle}{\sqrt{2}}$
11	ZX	$\|\Psi^-\rangle = \frac{\|01\rangle-\|10\rangle}{\sqrt{2}}$

Step 2: Alice sends her qubit to Bob. Now Bob holds both qubits in one of the four Bell states.

Step 3: Bob performs a Bell measurement (CNOT with A as control, B as target, then Hadamard on A, then measure both qubits in computational basis). The measurement outcome directly reveals Alice's two-bit message.

Outcome: Bob recovers Alice's message perfectly. One qubit transmission has conveyed two classical bits, enabled by the pre-shared entanglement.

7 Exercises

1. **Bell States:** Write all 4 Bell states in matrix form and verify they are:
   (a) Normalized
   (b) Mutually orthogonal
   (c) Maximally entangled
2. **Tensor Products:** Calculate:
   (a) $|0\rangle \otimes |+\rangle$
   (b) $|+\rangle \otimes |-\rangle$
   (c) $|1\rangle \otimes |1\rangle$

   Verify each result is normalized.
3. **CNOT Gate:** Apply CNOT to:
   (a) $|00\rangle$
   (b) $|10\rangle$
   (c) $|+0\rangle$
   (d) $|++\rangle$

   Which inputs create entanglement?
4. **Teleportation:** If Alice measures 10 during teleportation:
   (a) What is Bob's state before correction?
   (b) What gate must Bob apply?
   (c) What is the final state?
5. **Superdense Coding:** Alice wants to send "11":
   (a) What gates does she apply?
   (b) What is the resulting Bell state?
   (c) How does Bob decode "11"?
6. **Entanglement Detection:** Determine if these states are entangled:
   (a) $\frac{|00\rangle+|11\rangle}{\sqrt{2}}$
   (b) $\frac{|00\rangle+|01\rangle}{\sqrt{2}}$
   (c) $\frac{|01\rangle+|10\rangle+|11\rangle}{\sqrt{3}}$

   Justify your answers.
7. **Code Extension:** Modify the Bell state code to:
   (a) Create all 4 Bell states dynamically

   (b) Verify orthogonality numerically
   (c) Calculate entanglement entropy for each

8. **Physical Implementation:** Research one entanglement generation method:
   (a) Spontaneous parametric down-conversion (SPDC)
   (b) Superconducting qubit coupling
   (c) Trapped ion Coulomb interaction
   (d) Quantum dot emission

   Explain how entanglement is physically created.

8 Key Physical Concepts

This section summarizes the essential concepts from Phase 2. Keep this as a quick reference:

- **Tensor product**: Mathematical framework for composite quantum systems. When two quantum systems combine, their joint state lives in the tensor product of their individual spaces. This structure enables entanglement and is responsible for the exponential growth of quantum state space (2^n dimensions for n qubits).

- **Entanglement**: The most distinctly quantum phenomenon, with no classical analog. Entangled states cannot be factored into individual subsystem descriptions. Measurement outcomes are correlated more strongly than classical physics allows, violating Bell inequalities. Entanglement is a *resource* consumed by quantum protocols.

- **Bell states**: The four maximally entangled two-qubit states that form an orthonormal basis:
$$|\Phi^\pm\rangle = \frac{|00\rangle \pm |11\rangle}{\sqrt{2}}, \quad |\Psi^\pm\rangle = \frac{|01\rangle \pm |10\rangle}{\sqrt{2}}$$
  They are interconvertible via single-qubit gates and serve as the standard resource for teleportation, superdense coding, and quantum cryptography.

- **Multi-qubit gates**: Operations like CNOT, CZ, and SWAP that act on multiple qubits. CNOT is universal when combined with single-qubit gates—it can create entanglement from product states and is the workhorse of quantum circuits. SWAP decomposes into 3 CNOTs, showing CNOT's primacy.

- **Teleportation**: A protocol that transfers a quantum state using entanglement and two classical bits. The original state is destroyed (no-cloning preserved), and the protocol requires classical communication (preserving causality). Foundation for quantum networks and distributed computing.

- **Superdense coding**: The dual of teleportation: sending two classical bits using one qubit and pre-shared entanglement. Without entanglement, one qubit carries at most one classical bit (Holevo's bound). With entanglement, capacity doubles.

- **Ancilla measurement**: Physical implementation of quantum measurement as unitary evolution on a larger system (system + ancilla) followed by readout. Real detectors work this way: superconducting qubits (resonator), trapped ions (fluorescence), quantum dots (charge sensor). Reveals that measurement is not fundamentally different from other quantum operations.

9 Why Multi-Qubit Physics Matters

Everything we've covered in this chapter—tensor products, entanglement, Bell states, multi-qubit gates, teleportation, and superdense coding—forms the foundation of quantum computing. Without multi-qubit phenomena, quantum computers would be nothing more than expensive random number generators.

Entanglement is the resource that enables exponential speedups. Shor's algorithm for factoring, Grover's search algorithm, and quantum simulation all rely on creating and manipulating highly entangled states. The tensor product structure is what gives quantum computers their exponentially large state space, allowing them to explore solution spaces that would take classical computers longer than the age of the universe.

Teleportation and superdense coding aren't just clever tricks—they're blueprints for quantum networks. A future quantum internet will use teleportation to distribute quantum states between nodes, enabling distributed quantum computing and unhackable communication via quantum key distribution.

Understanding multi-qubit physics is understanding what makes quantum computers powerful. It's the difference between knowing how to turn on a computer and understanding how transistors work. You can use quantum computers without this knowledge, but to push the boundaries of what's possible—to design new algorithms, build better error correction, or create novel quantum protocols—you need to understand the physics at the foundation.